

Relational database

SQLite

CS 22B, Spring 2026

Module 08: Relationship Database

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA

- Relational Database
- SQL
- SQLite

Relational database: type of database that stores data related to each other in a pre-defined way. The relationships are identified at the time of creation and are between different entities or groups of data

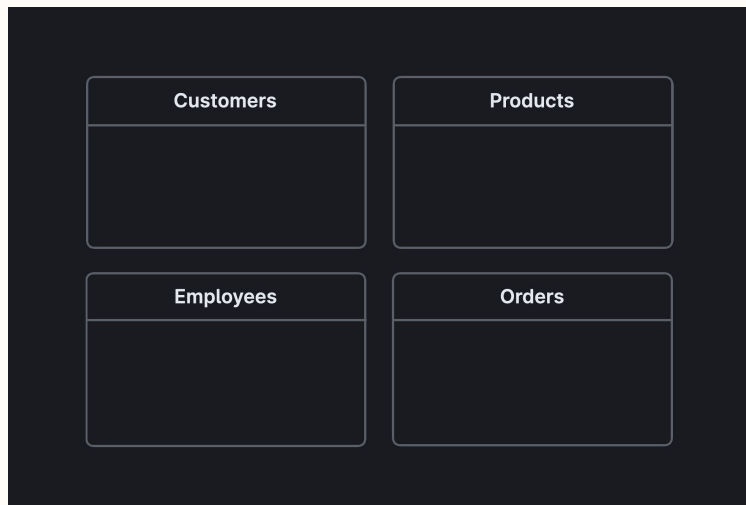
The anatomy of a relational database:

- **Tables:** Data representing an entity organized into columns in rows
 - row: record stored in table
 - columns: represent specific property
- **Properties:** attributes that are stored about an entity
- **Relationships:** relationships between tables
- **Indexes:** For connecting tables and quick look-up

Retailer database

- Track customers spending amount and time of purchase
- Track top customer to reward with gift at 1 year anniversary
- Make smart recommendations based on attributes such as age
- Track best-performing employees with highest sales with raise
- Store products and include a category and price property

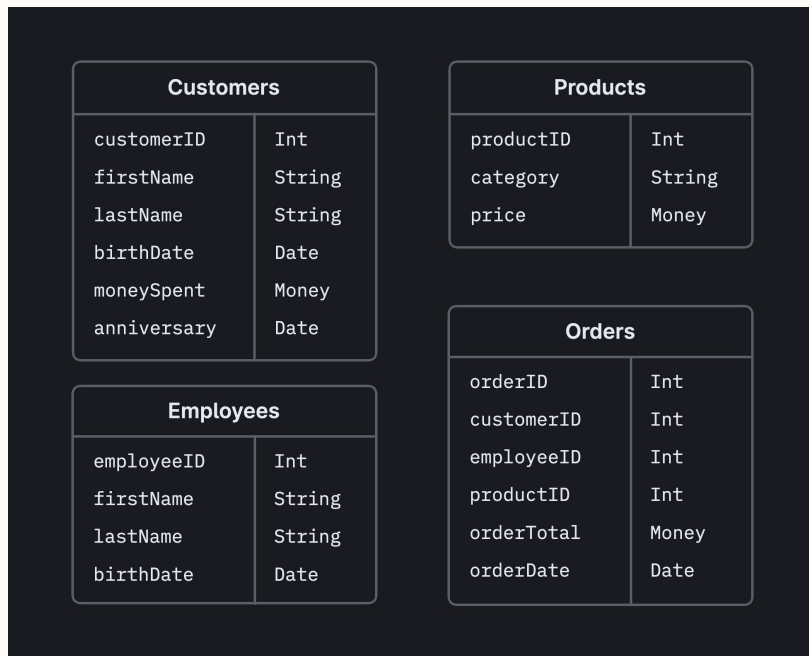
Define entities



Define properties to store in entities:

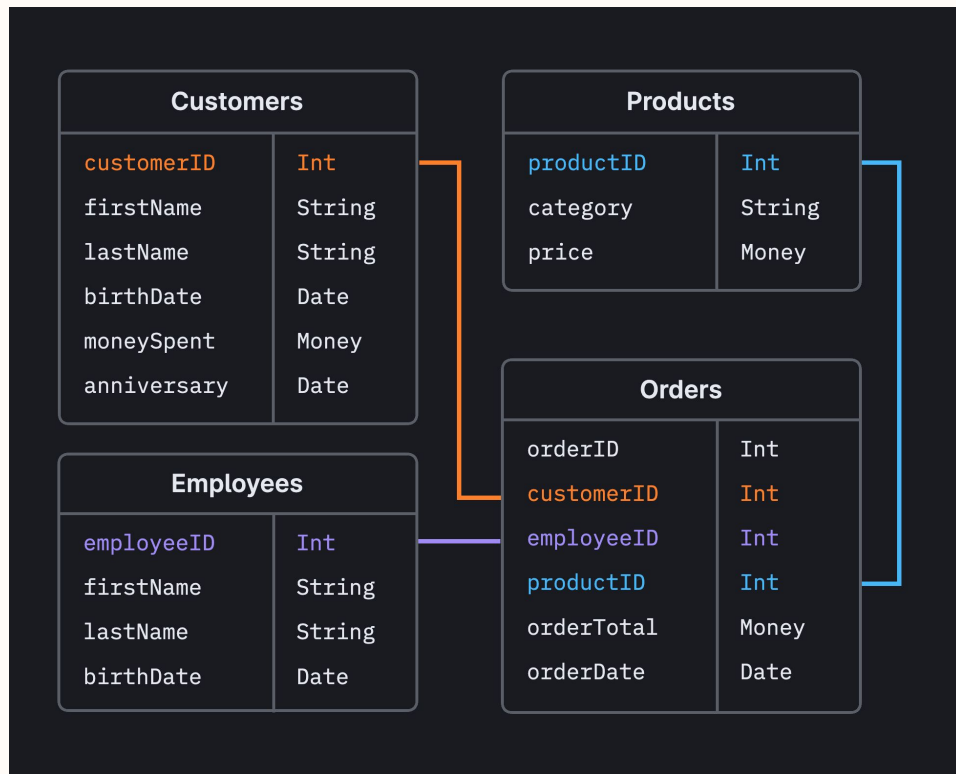
- Each entity has to have one unique identifying property
 - Unique value is primary key
 - Track top customer to reward with gift at 1 year anniversary

Define properties for entities



Define relationships

- Define how tables relate to each other
- Parent table will have unique primary key column.
- Child table will have its own primary key and parent_id that reference parent table
- Relationship between primary keys and foreign keys create relationship between tables



Structured query language (SQL) - a programming language for storing and processing information in a relational database.

SQLite is C library that enables us to access database using a variant of SQL query language



- SQLite is optimized for use in small environments such as mobile apps
- Does not require a separate server process to be run
- Integrated with Python using a module called sqlite3

What do you do with SQL?

08

SQL can:

- execute queries against a database
- retrieve data from a database
- insert records in a database
- update records in a database
- delete records from a database
- create new databases
- create new tables in a database
- create stored procedures in a database
- create views in a database
- set permissions on tables, procedures, and views

Class exercise

09

Scenario:

You want to create a database to track students. Go through the steps of setting up a relational database to think what we may want in the database

Step 1: Understand the problem

Step 2: Define entities

Step 3: Define properties

Step 4: Set up the relationships between entities

Connect to SQLite to create database

Which libraries?

```
import sqlite3 ← SQL library
from pathlib import Path ← Path to create database
```

Create a database connection

```
Path(gdrive+'student.db').touch()
```

* note that you have to mount your drive first

```
conn = sqlite3.connect(gdrive+'student.db') # creates a connection to the database
c = conn.cursor() # the cursor is what we'll actually use
```

Data types in SQLite database

There are 5 data types for values stored in in an SQLite database:

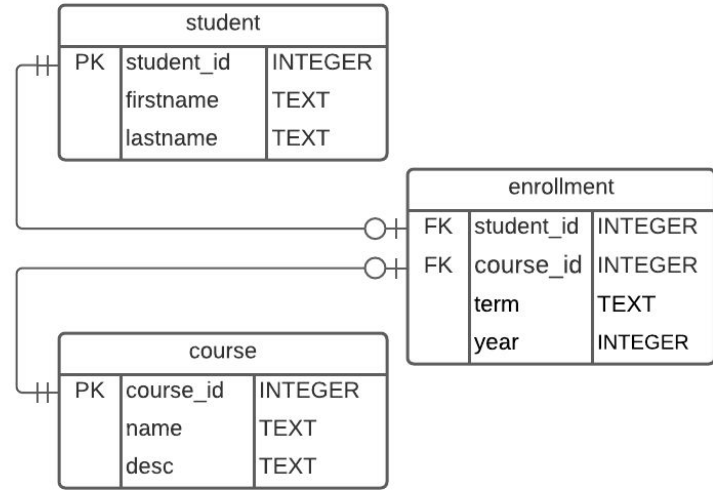
- NULL - The value is a NULL value.
- INTEGER - The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
- REAL - The value is a floating point value, stored as an 8-byte IEEE floating point number.
- TEXT - The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE).
- BLOB - The value is a blob of data, stored exactly as it was input.

See <https://www.sqlite.org/datatype3.html> for more information

Data types in SQLite database

Let's create a database to store the information about students and the courses that they enroll in.

- The student table that stores student information.
- The course table that stores course information.
- The enrollment table that stores the relationship between students and courses.



Created with Lucidchart

<https://sjsu.edu/it/services/applications/lucidchart.php>

Connect to SQLite to create database

13

Creates a connection to the database

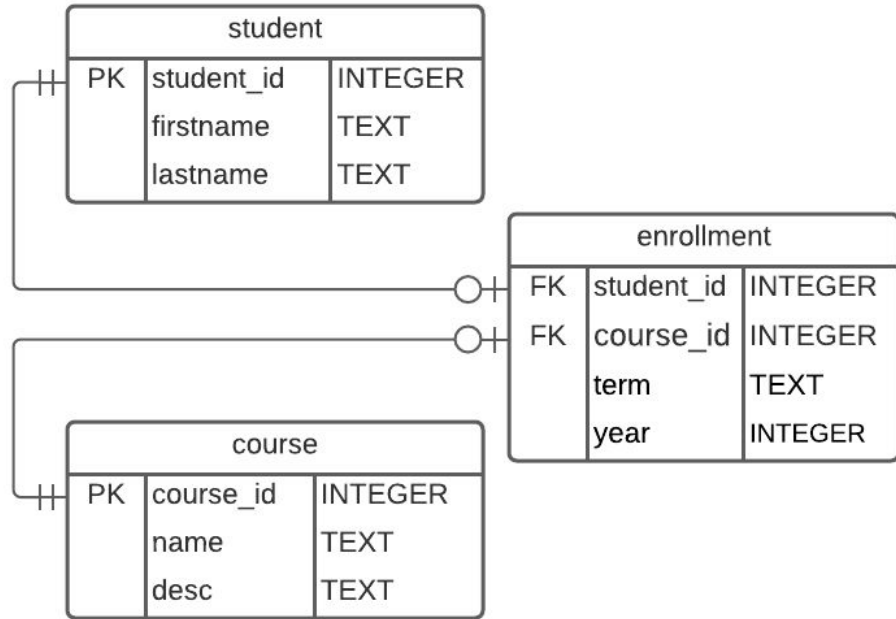
```
conn = sqlite3.connect(gdrive+'student.db')  
c = conn.cursor()           ←—— Cursor we will use
```

Creates an entity with 3 columns

```
SQL_CreateTable = '''CREATE TABLE IF NOT EXISTS  
student (student_id INTEGER PRIMARY KEY, ←—— Must be unique  
         firstname TEXT NOT NULL,  
         lastname TEXT NOT NULL  
         )'''  
  
c.execute(SQL_CreateTable)  
conn.commit()
```

Using FOREIGN KEY

14



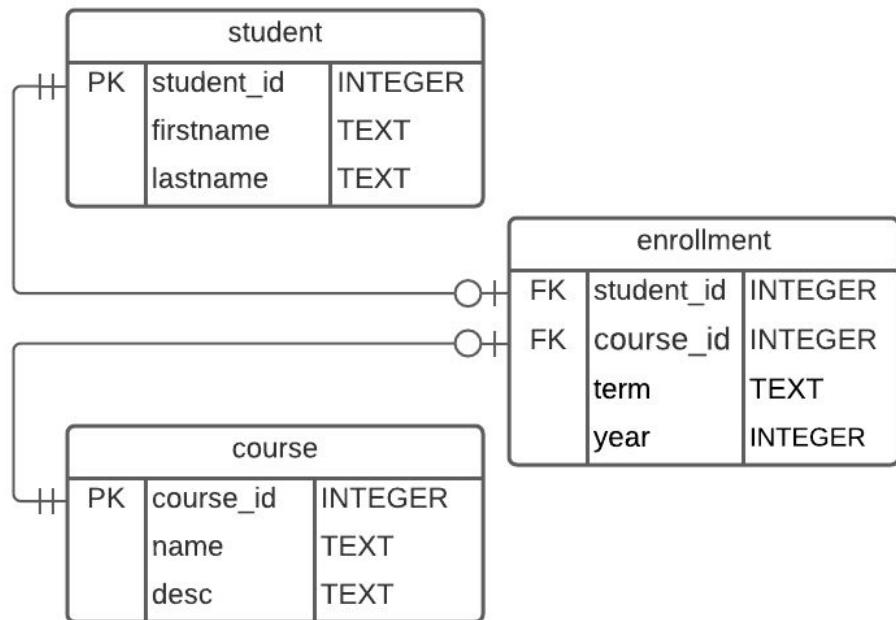
FOREIGN KEY creates relationship between two tables

- Value exist in another table

```
FOREIGN KEY (student_id)
REFERENCES student
(student_id)
ON DELETE CASCADE
ON UPDATE NO ACTION,
```

Review

15



PRIMARY KEY - Unique id that identifies who you are

FOREIGN KEY - relationship between tables. Who do you belong to

CASCADE DELETE - If parent removes, child goes too

NO ACTION UPDATE - You cannot change identity because it will break relationship

Class exercise

16

Scenario:

Your table uses the following

```
FOREIGN KEY (course_id)
  REFERENCES course
(course_id)
  ON DELETE CASCADE
```

What happens to enrollment if a course is deleted?

```
CREATE TABLE student ( student_id INTEGER, firstname TEXT,  
lastname TEXT, PRIMARY KEY (student_id) );
```

```
CREATE TABLE enrollment ( student_id INTEGER, course_id INTEGER,  
term TEXT, year INTEGER );
```

```
CREATE TABLE course ( course_id INTEGER, name TEXT, desc TEXT,  
PRIMARY KEY (course_id) );
```